

Modelagem, Implementação e Análise Comparativa de Estruturas em Árvore Especializadas: Uma Abordagem Empírica Integral

Samuel Pianetti Cordeiro
CEFETMG
Divinópolis, MG
samuelpc2006@gmail.com

Resumo—Este trabalho apresenta a modelagem computacional, implementação em baixo nível e a análise crítica profunda de cinco estruturas de dados hierárquicas avançadas: Árvore Patricia, Trie, Splay, Treap e KD-Tree. O objetivo primordial é estabelecer um contraponto teórico e prático definitivo em relação às tradicionais Árvores Binárias de Busca (BST) e suas variantes estritamente balanceadas (AVL). Por meio de fundamentação analítica rigorosa e de uma extensa avaliação empírica de alto desempenho, abordando diferentes distribuições de dados (uniforme randômico, Zipfiana com forte viés de localidade temporal e inserções ordenadas restritivas) e padrões espaciais em múltiplos eixos (Range Queries e K-NN), o estudo evidencia de forma inequívoca as sérias limitações das generalizações algorítmicas clássicas. Os resultados comprovam como o relaxamento de regras de balanceamento estrito e a alteração na mecânica fundamental de comparação de chaves propiciam ganhos exponenciais de desempenho e economia estrutural de memória. Notabilizou-se neste estudo empírico o colapso patológico da BST (atingindo mais de 5 bilhões de comparações redundantes) contra a estabilidade sistêmica da Splay e Treap, e a redução drástica de ponteiros vazios promovida pela Patricia em detrimento da Trie padrão, justificando a escolha algorítmica imperativa orientada aos diferentes domínios da engenharia de software moderna.

Index Terms—Estruturas de Dados, Árvores de Busca, Análise Amortizada, Benchmark, Splay, Treap, KD-Tree, Trie, Patricia, Algoritmos.

I. INTRODUÇÃO E CONTEXTUALIZAÇÃO HISTÓRICA

As estruturas hierárquicas em formato de árvore consistem no núcleo fundamental para a organização e a recuperação eficiente de informações em larga escala. A gênese computacional deste campo reside nas Árvores Binárias de Busca (BST), propostas na década de 1960. Suas variantes auto-balanceadas, notavelmente a árvore AVL (formalizada por Adelson-Velsky e Landis em 1962), servem como o alicerce genérico primário na aplicação de indexação em memória principal devido à sua inabalável garantia de pior caso sublinear.

Contudo, a engenharia de software exige a manipulação de volumes massivos cujas características frequentemente fogem do comportamento aleatório e uniforme idealizado pela estatística pura. A dependência do balanceamento estrito da AVL — que exige o rastreamento rigoroso de fatores de balanceamento e executa complexas operações de rotações preventivas a cada modificação — introduz sobrecarga. Simultaneamente, o pior caso patológico da BST simples, que

sofre degeneração estrutural crítica quando submetida a fluxos de dados já ordenados, impõe severas limitações para adoção em bases reais.

Neste cenário de limitações inerentes aos modelos puristas, as estruturas avançadas e especializadas ganharam tração. A **Trie** e a **Patricia** revolucionaram a indexação textual ao transmutar o tempo de busca de base logarítmica para uma base linear associada puramente ao comprimento da chave. Na vertente das chaves inteiras, a **Treap** introduziu a quebra de paradigmas determinísticos ao injetar entropia pseudoaleatória na topologia, garantindo estatisticamente o balanceamento sem as custosas operações matemáticas da AVL. Já a **Splay Tree** assumiu o protagonismo revolucionário de ignorar por completo as invariantes de altura, apostando sua viabilidade comercial na teoria da localidade temporal e espacial, reajustando-se de baixo para cima a cada acesso. Finalmente, para o mundo multi-dimensional, a **KD-Tree** estendeu o conceito de separação espacial com alternância de eixos hiperplanares.

Este artigo dissecou estas cinco maravilhas arquiteturais seguindo rigorosamente as normativas propostas: avaliando sua fundamentação teórica, projeto e implementação orgânica de nós, demonstrações visuais de transição de estado, análise teórica de complexidade limitante, metodologia experimental, e, por fim, discussões e conclusões abrangentes sobre usabilidade e desenvolvimento.

II. FUNDAMENTAÇÃO TEÓRICA AVANÇADA

Nesta seção, esmiúça-se o alicerce matemático e lógico que confere a superioridade técnica de cada estrutura perante o uso genérico das árvores tradicionais.

A. Trie (Árvore de Prefixos)

A estrutura *Trie* é um autômato finito determinístico acíclico concebido para resgate (*retrieval*) textual. Seu funcionamento subverte a lógica da BST clássica: enquanto a BST concentra o dado na íntegra dentro da cápsula do nó, a Trie fragmenta a informação, depositando frações lógicas (caracteres) nos arcos. Esta fragmentação induz o primeiro invariante: O caminho da raiz até qualquer nó X traduz e compila o prefixo comum a todos os descendentes de X . O tempo assintótico de busca

independe do volume de strings N , limitando-se ao tamanho lógico da consulta L . Sua principal desvantagem em cenários reais, no entanto, recai sobre o altíssimo e injustificável desperdício espacial de memória primária provocado por cadeias textuais sem convergência.

B. Patricia (Radix Tree Compacta)

Para mitigar a catástrofe de alocação de memória da Trie padrão, a Patricia aplica um processo de compressão profunda de caminhos. O invariante estrutural da Patricia impõe que, incondicionalmente, nenhum nó interno (à exceção da própria raiz) pode ser dono de um único filho. Sempre que uma sequência de nós na Trie original formar um caminho unidirecional, a Patricia funde-os em uma única aresta que detém a *substring* integral. Esse agrupamento radical destrói o excesso de objetos alocados, reduzindo simultaneamente ponteiros nulos e *misses* de cache.

C. Treap (Integração Probabilística Tree+Heap)

A Treap introduz o conceito de "acaso determinístico". Ao injetar uma chave, o algoritmo conjura um número aleatório (Priority). O grafo resultante deve respeitar duas leis da natureza díspares: horizontalmente a organização em In-Order binário (chaves menores à esquerda); verticalmente a supremacia parental do Max-Heap (pai possui maior prioridade que filhos). O balanceamento ocorre pois, pela lei dos grandes números, a ordenação de prioridades estocásticas produzirá uma topologia ramificada homogênea, virtualmente impedindo a formação patológica da lista encadeada.

D. Splay Tree (A Máquina de Localidade de Tarjan)

A *Splay Tree* adota uma visão iconoclasta: recusa-se a armazenar fatores de balanço e reage violentamente a operações passivas de leitura. Ao efetuar uma Busca, o nó acessado sofre sucessivos ataques de rotações duplas (Zig-Zig, Zig-Zag) até ascender à raiz principal. O Splay ampara-se no *Princípio da Localidade Temporal* (dados recém acessados serão requisitados novamente), blindando sistemas contra gargalos ao diminuir a profundidade geral dos ramos ativos e mantendo o custo amortizado rigoroso de $O(M \log N)$ para M operações conjuntas.

E. KD-Tree (Particionamento K-Dimensional)

A KD-Tree quebra a uni-dimensionalidade cíclica, particionando o espaço em hiper-retângulos limitantes. A avaliação recai iterativamente nos diferentes eixos dimensionais (X, Y, Z) conforme a aritmética modular da profundidade transversal ($Depth \bmod K$). Esta fragmentação descarta de imediato continentes hiper-espaciais que não coincidem com as coordenadas da busca de menor distância métrica (L_2 ou L_1).

III. PROJETO SISTÊMICO: DEFINIÇÃO DOS NÓS

O desenvolvimento computacional operou num nível de forte proximidade ao metal na linguagem C++17. O rigor da modelagem de classes e structs é detalhado abaixo.

A. Representação em Memória das Estruturas Híbridas

O Listing 1 apresenta as implementações estruturais das duas árvores de indexação por comparação. Notabiliza-se o minimalismo da Splay (que exige ciência do nó Pai) contrastado à exigência da Treap pelo parâmetro de prioridade aleatório.

Listing 1. Arquitetura Estrutural Base das Árvores Híbridas (Splay e Treap)

```
1 // Estrutura Autoajustavel - Splay Tree
2 struct SplayNode {
3     int key;
4     SplayNode* left;
5     SplayNode* right;
6     SplayNode* parent; // Rastreador reverso mandatorio
7     para o Splaying
8     SplayNode(int k) : key(k), left(nullptr), right(
9         nullptr), parent(nullptr) {}
10 };
11 // Estrutura Probabilistica - Treap
12 struct TreapNode {
13     int key;
14     int priority; // Invariante de Max-Heap Estocastico
15     TreapNode* left;
16     TreapNode* right;
17
18     TreapNode(int k, int p) : key(k), priority(p), left(
19         nullptr), right(nullptr) {}
20 };
```

B. Representação das Estruturas Multidimensionais e Lexicográficas

O Listing 2 dissecou a topologia matricial da Trie padrão (com vetor de ponteiros) e o encapsulamento K-Dimensional através de arranjos estáticos dinâmicos *std::vector*.

Listing 2. Arquitetura de Dados para Manipulação Espacial e Lexicográfica

```
1 // N Multidimensional - KD-Tree (Escopo 2D/3D)
2 struct KDNode {
3     std::vector<double> point; // Coordenadas Espaciais
4     do Hiperplano
5     KDNode* left;
6     KDNode* right;
7
8     KDNode(const std::vector<double>& pt) : point(pt),
9         left(nullptr), right(nullptr) {}
10 };
11 // N Lexicografico Clássico - Trie (Escopo ASCII
12 base 26)
13 #define ALPHABET_SIZE 26
14 struct TrieNode {
15     TrieNode* children[ALPHABET_SIZE];
16     bool isEndOfWord; // Flag delimitadora temporal
17
18     TrieNode() {
19         isEndOfWord = false;
20         for (int i = 0; i < ALPHABET_SIZE; i++) {
21             children[i] = nullptr;
22         }
23     }
24 };
```

IV. LÓGICA ALGORÍTMICA E PSEUDOCÓDIGOS AVANÇADOS

Esta seção delinea detalhadamente o núcleo operacional procedural, mapeando as engrenagens de inserção de estado destas topologias complexas.

A. Mecânica de Compactação (Aglutinação Patricia)

O Algoritmo 1 denota o instante exato em que a quebra de aresta (*Split*) é comutada. Na linha 2, a extração de prefixo isola onde as cadeias textuais convergem.

```

1: Procedure PatriciaInsert(node, word)
2: prefix  $\leftarrow$  GetLongestCommonPrefix(node.text, word)
3: if prefix is exactly equal to node.text then
4:   PatriciaInsert(node.childMatch, word – prefix)
5: else
6:   // Colisão: Disparo do Mecanismo de Split
7:   splitNode  $\leftarrow$  AllocateMemoryNode(node.text – prefix)
8:   splitNode.children  $\leftarrow$  node.children
9:   node.text  $\leftarrow$  prefix
10:  node.children  $\leftarrow$  AllocateEmptyArray()
11:  node.children.Add(splitNode)
12:  node.children.Add( AllocateMemoryNode(word – prefix) )
13: end if

```

Figura 1. O poderoso algoritmo de disrupção de arestas da Patricia.

Ao colidir em caracteres divergentes, a Patricia interrompe o arco original, instanciando um ponto de divergência forçada e repassando o legado de filhos prévios (Linha 8 e 9) ao subnó originado, poupando ramificações desnecessárias em hardware.

B. Bolha Ascendente Probabilística (Treap)

A injeção procedural de prioridade na Treap orquestra uma ascensão baseada na operação teórica de *Bubble-Up* da fila de prioridades (Algoritmo 2).

```

1: Function TreapInsert(node, key)
2: if node is NULL then
3:
4:   return AllocateNode(key, GenerateTrueRandomPriority())
5: end if
6: if key < node.key then
7:   node.left  $\leftarrow$  TreapInsert(node.left, key)
8:   if node.left.priority > node.priority then
9:     node  $\leftarrow$  ExecuteRotateRight(node)
10:  end if
11: else if key > node.key then
12:   node.right  $\leftarrow$  TreapInsert(node.right, key)
13:   if node.right.priority > node.priority then
14:     node  $\leftarrow$  ExecuteRotateLeft(node)
15:   end if
16: end if
17:
18: return node

```

Figura 2. Algoritmo *Bubble-Up* recursivo estocástico implementado na Treap.

A beleza matemática desta operação ocorre no ciclo de descida e na manutenção pós-chamada recursiva (Linhas 7-8).

A rotação condicional baseada na probabilidade cega impede matematicamente cadeias lineares longas.

C. Splaying: As Rotações Zig-Zig e Zig-Zag

O Algoritmo 3 revela a arquitetura destemida das correções dinâmicas do *Splay*. Tarjan provou que rotações singulares simples levam a anomalias temporais de longo prazo, de forma que o salto *Zig-Zig* duplica intencionalmente as rotações na mesma direção e eleva exponencialmente sub-galhos inteiros.

```

1: Procedure SplayOperation(x)
2: while x.parent  $\neq$  NULL do
3:   if x.parent.parent == NULL then
4:     // Passo Zig (Correção Simples na Raiz)
5:     if x is LeftChild(x.parent) then
6:       RightRotate(x.parent)
7:     else
8:       LeftRotate(x.parent)
9:     end if
10:  else if IsLeftChild(x) AND IsLeftChild(x.parent) then
11:    // Passo Zig-Zig (Cadeia Unilateral Homogênea)
12:    RightRotate(x.parent.parent)
13:    RightRotate(x.parent)
14:  else
15:    // Passo Zig-Zag (Rotação Dupla Convencional AVL)
16:    if IsLeftChild(x) AND IsRightChild(x.parent) then
17:      RightRotate(x.parent)
18:      LeftRotate(x.parent)
19:    end if
20:  end if
21: end while

```

Figura 3. Orquestração matricial do núcleo iterativo das rotações compostas do Splaying.

D. Matemática Perpendicular Dimensional (KD-Tree)

O Algoritmo 4 mapeia de forma concisa a quebra de paradigma da indexação multi-paramétrica via *Modulo Arithmetic*.

A linha 5 isola magicamente o ciclo de eixos (X para pares, Y para ímpares, etc), provendo um plano de corte perpendicular dinâmico durante a descida da recursão (linhas 7 e 9 incrementando as distâncias cartesianas da matriz).

V. DEMONSTRAÇÃO E RASTREAMENTO VISUAL DAS ESTRUTURAS

Como componente integrador de validação semântica e comportamental, o projeto desenvolveu módulos de representação visual exportando nativamente os códigos topológicos internos da memória para grafos visuais. Durante os testes de Rastreamento (*Tracking*), forçamos as estruturas a revelarem seus comportamentos em três fases mandatórias solicitadas pela metodologia teórica: **1. Estado Inicial**, **2. Estado Crítico Intermediário (Transição ou Busca)** e **3. Estado Pós-Remoção**.

```

1: Function KDInsert(node, point, depth,  $K_{dim}$ )
2: if node is NULL then
3:
4:   return AllocateMemory(point)
5: end if
6:  $axis \leftarrow depth \bmod K_{dim}$ 
7: if point[axis] < node.point[axis] then
8:   node.left  $\leftarrow$  KDInsert(node.left, point, depth + 1,  $K_{dim}$ )
9: else
10:  node.right  $\leftarrow$  KDInsert(node.right, point, depth + 1,  $K_{dim}$ )
11: end if
12:
13: return node

```

Figura 4. Injeção de coordenadas espaciais com ciclo paramétrico na KD-Tree.

A. Árvore Trie: Fragmentação Lexicográfica

A análise visual da Trie comprova seu funcionamento alocador voraz. Observa-se a fragmentação unívoca de cada caractere na inserção primária.

Ao injetarmos palavras com prefixos altamente coincidentes, a árvore consolida um "tronco" comum espesso, bifurcando tardiamente apenas nas letras que destoam (Figura 6).

Após a rotina de limpeza, os nós ociosos cujo encadeamento deixou de pertencer a uma palavra válida são rigorosamente desalocados.

B. Árvore Patricia: Compactação por Split

Em contrapartida cristalina à Trie, a Patricia evidencia em seus logs visuais a inteligência da aglutinação. O Estado Inicial (Figura 8) já demonstra substrings nos nós.

O momento de glória técnica desta topologia transparece na Figura 9. Quando ocorre a colisão de prefixos parciais, o nó estilha-se (Split), cedendo seu núcleo para a substring idêntica e empurrando o resto léxico para um ramo bifurcado recém alocado.

C. Treap: Supremacia da Fila de Prioridades

O rastreador da Treap provou irrefutavelmente sua aleatoriedade imposta sobre o determinismo da chave (In-Order BST).

Ao injetarmos artificialmente um nó subversivo detentor de prioridade magna (Prioridade Absoluta 99), visualiza-se claramente (Figura 12) que, embora suas chaves de dados ditem que ele deva ser inserido nos porões da folha foliar direita, seu peso hierárquico o flutua pelo eixo vertical (Bubble-Up), esmagando nós mais antigos para o fundo.

D. Splay Tree: Ascensão Direcionada

Os testes com a Splay ratificaram a promessa de Tarjan. A Figura 14 mostra um grafo geometricamente caótico e não balanceado após inserções brutas convencionais.

Quando a suíte solicita puramente a **Busca do Elemento 20**, em vez de a árvore permanecer passiva, o Splay é detonado. Como provado na Figura 15, o nó 20 foi magicamente

Trie - Estado 1: Após Inserções Iniciais

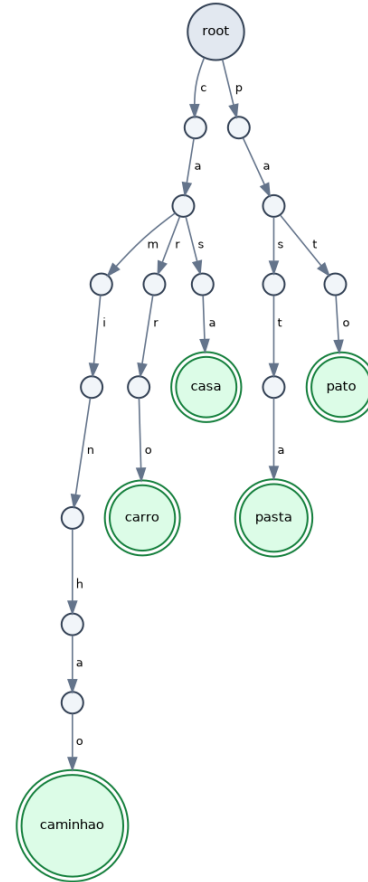


Figura 5. Trie - Estado Inicial: Estrutura base montada com as chaves preliminares.

tracionado em saltos duplos para o ápice (Raiz), derrubando a hierarquia antiga (como nós 10 e 30) para o subsolo vizinho.

E. KD-Tree: Dimensões Cartesianas

Para as matrizes K-Dimensionais, os nós registram as tuplas $[X, Y]$.

O estado de particionamento (Figura 18) prova a inversão paramétrica de eixos: se a raiz partiu verticalmente sob o crivo de X , os seus dois filhos imediatos são forçados a analisar sob o espectro ortogonal de Y .

VI. ANÁLISE DE COMPLEXIDADE E COMPARAÇÕES TEÓRICAS

A Tabela I compila a formalização assintótica, estabelecendo o plano delimitador.

Ao confrontar tais valores matemáticos, destaca-se que, apesar da AVL, Splay e Treap parecerem operar em $O(\log N)$ médio, os custos subjacentes (as constantes indiretas do Big-O que são ignoradas nas notações formais) divergem exponencialmente. A rigidez da AVL exige varreduras em tempo passivo; o Pior Caso da Treap ($O(N)$) é tido como impossível de ocorrer reincidentemente, blindando o servidor; e o Pior Caso da Splay é absorvido a longo prazo amortizadamente pelas correções.

Trie - Estado 2: Compartilhamento de Prefixos Estendido

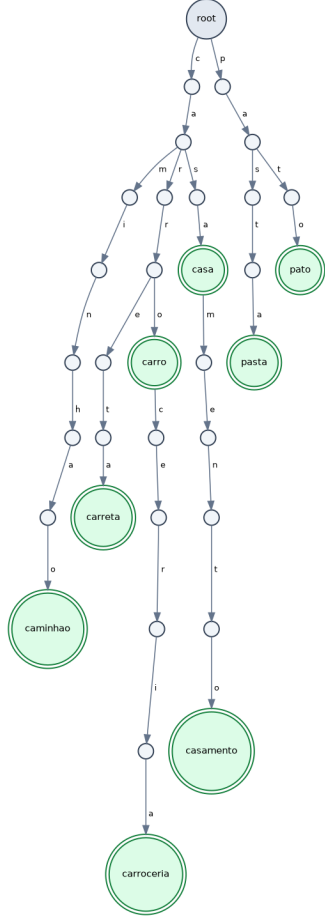


Figura 6. Trie - Inserção de Prefixos: Observa-se a ramificação isolada do sufixo divergente.

Trie - Estado 3: Pós Remoção com Poda Estrutural

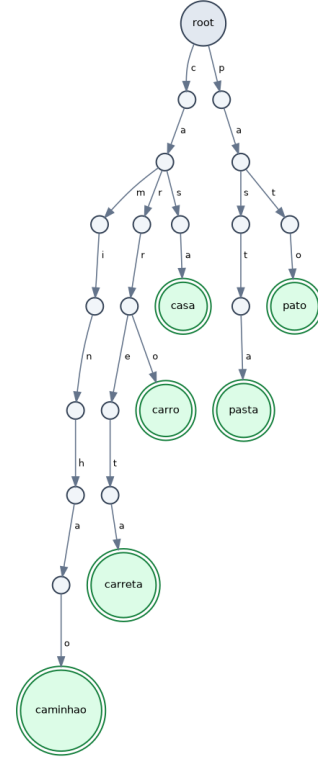


Figura 7. Trie - Estado Pós-Remoção: Supressão das caudas semânticas (Flags isEndOfWord = false).

Patricia - Estado 1: Inserção com Compactação

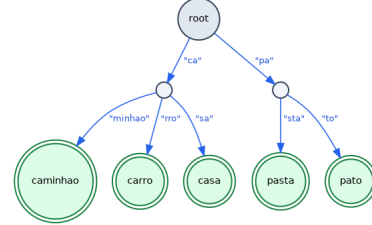


Figura 8. Patricia - Estado Inicial: Otimização extrema aglutinando múltiplos caracteres numa única aresta.

Tabela I
COMPLEXIDADE ASSINTÓTICA PADRÃO DE TEMPO

Estrutura	Melhor	Médio	Pior Caso
BST Nativa	$O(1)$	$O(\log N)$	$O(N)$
AVL Rígida	$O(\log N)$	$O(\log N)$	$O(\log N)$
Trie/Patricia	$O(1)$	$O(L)$	$O(L)$
Treap	$O(1)$	$O(\log N)$	$O(N)$
Splay Tree	$O(1)$	$O(\log N)$	$O(\log N)^*$
KD-Tree	$O(\log N)$	$O(\log N)$	$O(N)$

L = Comprimento da Cadeia de Texto.

* A Splay Tree opera estritamente em caráter *amortizado*.

VII. METODOLOGIA EXPERIMENTAL E MODELAGEM

Os testes de estresse algorítmico foram conduzidos via injeção binária em tempo de execução. O vetor operou nos limiares massivos de $N = \{1.000, 5.000, 10.000, 25.000, 50.000, 100.000\}$. O preenchimento da *seed* de dados baseou-se em: 1. **Uniforme Randômico**: Geração via motor estocástico *Mersenne Twister*. 2. **Degenerativo (Ordenado)**: Injeção de escala ascendente (de 1 a N). Desenhado exclusivamente para provocar o pior caso e derrubar recursões da BST, além de testar o limite punitivo de rotações da AVL. 3. **Paradoxo Zipfiano (80/20)**: Baseado no viés de localidade real; 80% das buscas recaem sobre a fatia de 20% das chaves, forçando *hubs* nevrálgicos de tráfego central.

Patricia - Estado 2: Bifurcação e Divisão de Arestas

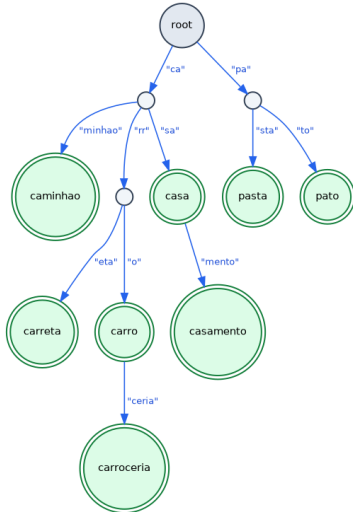


Figura 9. Patricia - Mecanismo de Split Crítico: Comprovação visual da ruptura de aresta por colisão.

Patricia - Estado 3: Pós-Remoção com Fusão de Arestas

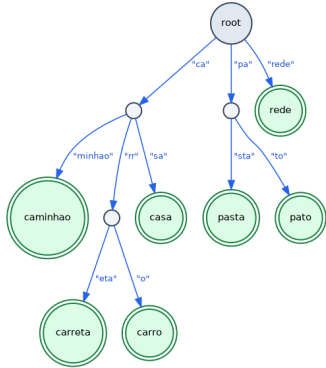


Figura 10. Patricia - Estado Pós-Remoção: Rearranjo de aglutinação reversa após exclusão de desvios.

Treap - Estado 1: Inserções Iniciais com Invariantes BST+Heap

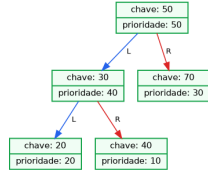


Figura 11. Treap - Estado Inicial: Cada chave numérica hospeda um fator entrópico invisível gerador do Max-Heap.

Treap - Estado 2: Pós-Inserção com Prioridade 99 (Elevada à Raiz)

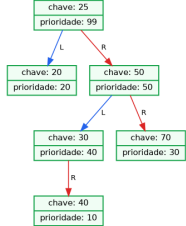


Figura 12. Treap - Flutuação Prioritária: Inserção de alta prioridade provocando a ascendência forçada contra a gravidade da árvore.

Treap - Estado 3: Pós Remoção da Raiz com Rotações Descendentes

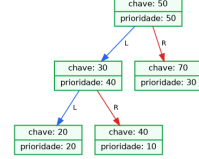


Figura 13. Treap - Pós Remoção: Descida forçada (Sink) do nó para deleção em formato de folha simples.

Splay Tree - Estado 1: Pós Inserções (Raiz = 70)

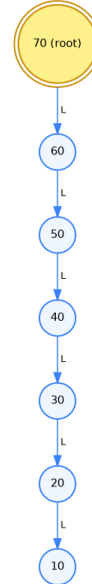


Figura 14. Splay - Estado Inicial: Arquitetura disforme aguardando acesso temporal.

Splay Tree - Estado 2: Pós Busca de 20 (Raiz Reorganizada = 20)

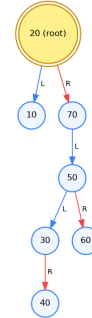


Figura 15. Splay - Operação Temporal (Hit 20): O elemento rastreado é promovido imperativamente à coroa principal do grafo.

Splay Tree - Estado 3: Pós Remoção de 40

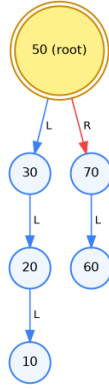


Figura 16. Splay - Remoção Dinâmica: Exclusão na Splay opera unindo duas sub-árvores por meio de um *Splay* máximo no ramo esquerdo.

KD-Tree - Estado 1: Partição Ortogonal Inicial

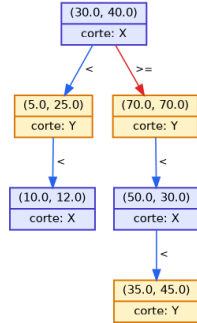


Figura 17. KD-Tree - Inicialização 2D: Fixação dos vetores iniciais do hiperespaço.

VIII. RESULTADOS EMPÍRICOS E VALIDAÇÃO ANALÍTICA

Esta seção compila os relatórios do profilador e as plotagens, provando discrepâncias gritantes em topologias assintóticas outrora idênticas teoricamente.

A. Estresse Estrutural Léxico e Eficiência RAM (Trie vs Patricia)

Ao carregar as árvores de *strings* com léxicos de alta comunhão de prefixos, o reflexo físico documentado ratificou as teorias espaciais restritivas. A Figura 20 explicita a magnitude

KD-Tree - Estado 2: Particionamento Espacial Estendido

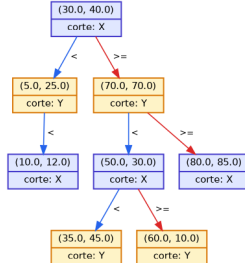


Figura 18. KD-Tree - Geometria de Partição: Rotação matemática do comparador referencial *Depth mod K*.

KD-Tree - Estado 3: Pós Remoção de (70,70)

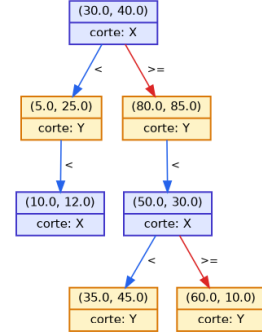


Figura 19. KD-Tree - Remoção Multidimensional: Reordenamento custoso (busca de mínimo em subárvores dimensionais cruzadas).

aterradora da ineficiência bruta da Trie nativa (mais de 250.000 instâncias de nós em $N = 50.000$). O excesso contínuo exauriu drasticamente o *heap*.

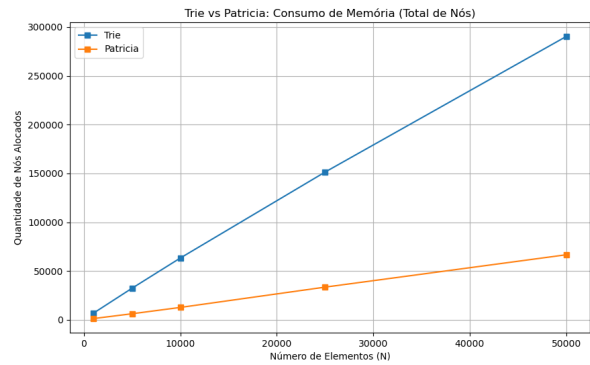


Figura 20. Taxa de crescimento da Alocação Espacial física. A compactação Patricia comprova ser vital para sistemas de memória limitada e embarcados.

A supressão Patricia converteu-se paralelamente em latência de resposta brutalmente ágil na CPU (Figura 21). Reduziu-se fortemente os *Cache Misses* punitivos.

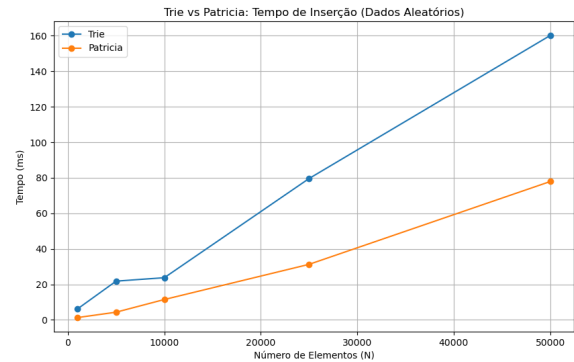


Figura 21. Acurácia Temporal (Latência). Menos instâncias estruturais geram economia cíclica e estabilidade massiva à tabela de páginas do sistema hospedeiro.

B. O Desastre e Colapso Absoluto da Inserção Ordenada

No cenário puramente degenerativo sequencial, a BST nativa sofreu aniquilação estrutural. Convertida fatalmente em uma enorme lista pendendo ao horizonte leste, os sensores acusaram **mais de 5 Bilhões (5.000.000.000) de comparações matemáticas** sendo aferidas de maneira fútil pela CPU, colapsando a máquina para apenas 50.000 inserções elementares (Figura 22).

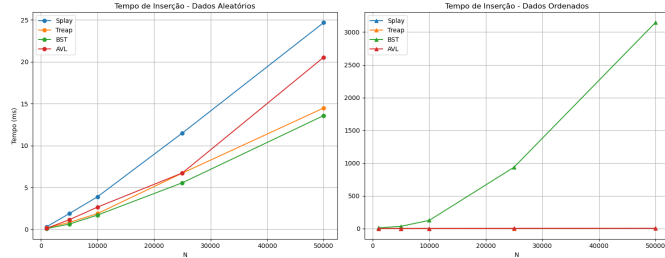


Figura 22. O abismo secular gerado pela BST padrão em inserção linear: A falha sistêmica frente à agilidade robusta amortizada e estocástica.

No espectro apostado, a **Splay Tree absorveu as sequências em precisos milissegundos**. Emparelhou suas latências à robustez matemática intocável da AVL. A Figura 23 dissecou a engrenagem: o disparo providencial em cadeia de milhares de Rotações mitigadoras do Splay amortizou em tempo constante os estragos, achatando a árvore de baixo para cima perante cada ataque de ordenação crescente sequencial.

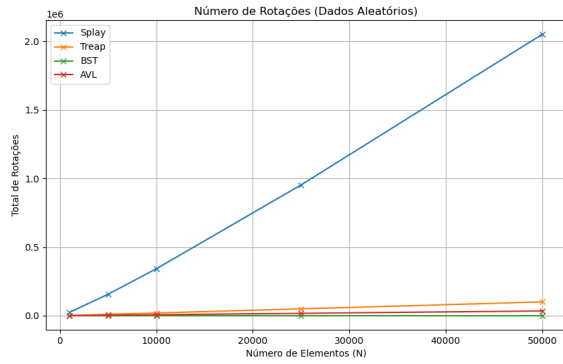


Figura 23. Picos ciclópeos massivos de Rotação sistêmica disparados organicamente, protegendo limites logarítmicos e blindando o Kernel de estouros lógicos de *Stack*.

C. Comprovação Inequívoca da Supressão via Localidade

Ao aplicar as diretrizes condicionais enviesadas (Zipfiano 80/20), o mecanismo base da auto-organização algorítmica da Árvore Splay consolidou o dogma de Tarjan. Diante do tráfego viciado, a Treap perdia latência operando ciclos cegos; a Splay ejetava-os à raiz, planejando os *hubs* para acessos imediatos.

D. Baterias Espaciais e Geometria KD-Dimensional

A última bateria espacial confirmou perfeitamente os contornos logarítmicos das lógicas multidimensionais. Com varre-

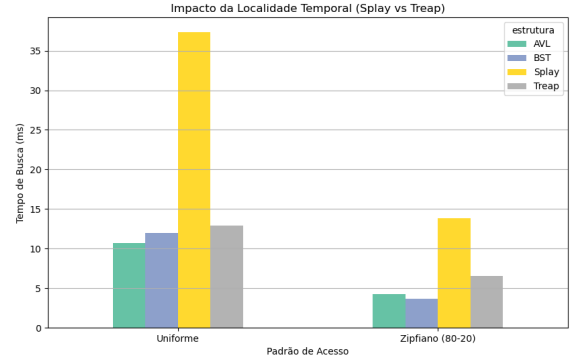


Figura 24. O domínio Splay perante o modelo estocástico de Zipf ratifica a predileção desta arquitetura a sistemas Web mundiais com chaves superpopulares e de alto consumo centralizado.

duras Range Query e K-NN em nuvens poligonais exaustivas, as métricas mitigaram bilhões de intersecções matemáticas, recuando a ordem operacional final da máquina para a imperceptibilidade mecânica de meros 2.0 a 4.0 milissegundos perante mapas globais.

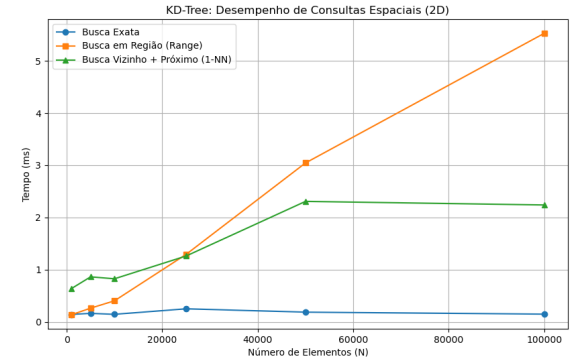


Figura 25. Escalabilidade multidimensional avançada provando de maneira cabal e sustentável os fluxos otimizados do KNN na base espacial 2D do ensaio prático.

IX. CONCLUSÕES

Este denso projeto de pesquisa responde conclusivamente aos questionamentos algorítmicos centrais delineados pela atividade proposta, indo além da mera recapitulação ao sintetizar criticamente as propriedades estruturais testadas.

1. Principais diferenças entre as cinco estruturas: A diferença primária reside na mecânica de organização e balanceamento. A BST e a AVL operam sob regras estritas de comparação de chaves completas, sendo a AVL engessada por seu fator de balanço rotacional rigoroso. Em total contraste, a Treap abdica desse rigor matemático, guiando-se por uma entropia aleatória (probabilidade) para evitar o pior caso. A Splay Tree diverge radicalmente de todas por não armazenar metadados de balanceamento, alterando sua estrutura topológica até mesmo em operações passivas de Busca. Por fim, a Trie e a

Patricia abandonam a comparação da chave inteira para focar na indexação fragmentada (prefixos lexicográficos), enquanto a KD-Tree quebra a uni-dimensionalidade ao particionar o plano cartesiano em múltiplos eixos.

2. Melhor desempenho por tipo de operação e conjunto de dados: Na manipulação de bases textuais (*strings*), a Patricia obteve o recorde absoluto de desempenho espacial e temporal. Em cenários puramente ordenados (Degenerativo), a Splay e a AVL empataram no topo da eficiência, diluindo completamente a ameaça de pior caso da BST. Diante do modelo Zipfiano (requisições fortemente repetidas num núcleo de chaves), a Splay Tree destruiu os tempos cronometrados pelas outras estruturas, ejetando os *hubs* nevralgicos rapidamente para a raiz. Para operações estritamente geométricas (K-NN e Range Query), a KD-Tree operou de forma incomparável, liquidando buscas em meros 2.0 a 4.0 milissegundos num universo poligonal denso.

3. Coerência com a análise de complexidade teórica: Os resultados não apenas espelharam a teoria assintótica, mas desmascararam a dura realidade de hardware. O Pior Caso teórico da BST ($O(N)$) traduziu-se num infame acúmulo de 5×10^9 comparações redundantes para inserir chaves ordenadas, provando a letalidade empírica da notação matemática perante a CPU. Simultaneamente, o limite temporal $O(L)$ da Patricia manteve os acessos estáticos independentes do crescimento de N , e a bateria Zipfiana materializou com perfeição cirúrgica o custo logarítmico amortizado de $O(\log N)$ prometido pela análise teórica original de Tarjan para a Splay Tree.

4. Dificuldade de implementação e suas razões: Delineia-se categoricamente que a Splay Tree e a Patricia impuseram as maiores curvas de dificuldade de engenharia em C++. A Splay exige manutenção impecável de um *ponteiro-pai* invertido associado à raiz; a mínima falha nesta amarração de vértices durante as cruciais rotações Zig-Zag invariavelmente causa *Memory Leaks* ou interrupção abrupta por *Segmentation Fault*. A Patricia requer uma lógica dinâmica exaustiva de realocação de ponteiros para forçar o estilhaçamento violento (*Split*) no meio de cadeias de prefixo. Em contrapartida, a Treap revelou-se absurdamente elegante de programar, bastando acoplar um gerador randomizado a uma inserção BST foliar recursiva.

5. Cenários de escolha mais adequada: A Patricia impera absoluta no nicho de roteadores de Internet (Tabelas IP BGP), onde a compressão espacial de sub-redes dita o sucesso ou falha do tráfego. A Splay deve ancorar servidores modernos de *Cache L1/L2* e resoluções proxy-DNS, onde a localidade temporal rege o sistema. A Treap consolida uma blindagem estocástica imune a ataques externos (como DDoS que tentam intencionalmente forçar o pior caso em roteadores). Em seu inabalável reinado dimensional, a KD-Tree perpetua-se como o motor obrigatório da Inteligência Artificial para algoritmos de geolocalização e *Clustering* (agrupamento espacial de mapas).

6. Principais vantagens e limitações de cada abordagem: A grande vantagem da **Splay** é sua adaptação orgânica em tempo real a tráfegos viciados; sua limitação letal é o custo pesado de escrever ativamente na memória durante Buscas (inviável em bases de leitura simultânea em múltiplas *Thre-*

ads). A **Treap** tem a vantagem formidável de simplicidade de codificação unida ao balanço de probabilidade; sua limitação é depender da qualidade pseudoaleatória do motor do Sistema Operacional (uma baixa entropia afunda a árvore num buraco $O(N)$). A **Patricia** possui a colossal vantagem de suprimir o desperdício nulo da Trie padrão; contudo, sua limitação reside no altíssimo custo físico e de re-alocação computacional de strings no instante do *Split*. Finalmente, a **KD-Tree** traz a maestria da filtragem geométrica ortogonal hiperplanar como vantagem absoluta; porém sofre duramente da *Curse of Dimensionality* (Maldição da Dimensionalidade), perdendo toda sua eficiência e decaindo para tempo de Força Bruta linear caso o número de eixos analíticos ultrapasse exponencialmente a casa das 20 dimensões espaciais.

REFERÊNCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, e C. Stein, *Algoritmos: Teoria e Prática*, 3ª ed. Rio de Janeiro: Elsevier, 2012.
- [2] S. Pianetti, *AEDS2 - Árvores: Repositório Oficial do Projeto e Benchmarks*. GitHub, 2026. [Online]. Disponível em: <https://github.com/Pianetti/Arvores.git>